



prompt2analytics: A Rust Toolkit for Econometrics and Data Analytics

First Author

University of Somewhere

Abstract

This paper introduces **prompt2analytics**, a comprehensive econometrics and data analytics toolkit implemented in pure Rust. The software provides a wide range of statistical methods including ordinary least squares regression with heteroskedasticity-consistent standard errors, panel data models (fixed and random effects), instrumental variables estimation, difference-in-differences, discrete choice models, time series analysis, and machine learning algorithms. Key design features include a pure Rust implementation without external BLAS/LAPACK dependencies, multiple interfaces (command-line, Model Context Protocol server, desktop application), and validated numerical accuracy against established R packages. Performance benchmarks demonstrate competitive speed compared to **data.table** and **dplyr** for data manipulation, and **lfe** and **plm** for econometric estimation.

Keywords: econometrics, Rust, regression, panel data, causal inference, time series, MCP, R.

1. Introduction

The statistical software landscape for econometrics has long been dominated by R (R Core Team 2024), Stata, and increasingly Python. R in particular has developed a rich ecosystem for econometric analysis, as documented by Zeileis and Koenker (2008) and more recently by Michelaki and Tsagris (2025), who provide a systematic overview of 207 econometrics-related R packages. The **AER** package (Kleiber and Zeileis 2008, 2024) has become a standard reference for applied econometrics in R. Alternative ecosystems have emerged in Julia with **Econometrics.jl** (Calderón 2020) and in MATLAB with dedicated toolboxes for panel data analysis (Álvarez, Barbero, and Zofío 2017). Comparative studies suggest that compiled languages offer substantial performance advantages for computationally intensive econometric applications (Aruoba and Fernández-Villaverde 2015; Duarte, Duarte, Fonseca, and Montecinos 2020).

While these environments offer mature, well-tested implementations of econometric methods, they present challenges for deployment in production systems, particularly those requiring high performance, memory safety, or integration with modern AI systems. Moreover, numerical accuracy remains a concern across statistical software platforms, as documented in a series of papers by McCullough and colleagues (McCullough 1997; McCullough and Vinod 1999; McCullough 1999), who demonstrated that users cannot safely assume their software is numerically accurate without verification.

The Rust programming language has emerged as a compelling platform for scientific computing, combining the performance characteristics of compiled languages like C++ with memory safety guarantees enforced at compile time (Rust Foundation 2024). The Rust data science ecosystem has matured rapidly, with **polars** providing high-performance DataFrame operations (Pola-rs Contributors 2024), Apache Arrow **DataFusion** enabling fast analytical query processing (Lamb, Shen, Heres, Chakraborty, Kabak, Hsieh, and Sun 2024), and **linfa** (rust-ml Contributors 2024) offering machine learning algorithms inspired by Python’s **scikit-learn**. However, a gap remains: no comprehensive econometrics library exists for Rust.

prompt2analytics addresses this gap by providing a comprehensive econometrics toolkit implemented in pure Rust. The software is built on **polars** (Pola-rs Contributors 2024) for DataFrame operations, **ndarray** (ndarray Contributors 2024) for n-dimensional arrays, **faer** (el and faer Contributors 2024) for linear algebra, and **statsr** (statsr Contributors 2024) for statistical distributions.

The key features of **prompt2analytics** include:

- Pure Rust implementation of regression, panel data, instrumental variables, discrete choice, and time series methods
- Heteroskedasticity-consistent standard errors (HC0–HC3) following White (1980) and MacKinnon and White (1985)
- High-dimensional fixed effects estimation following Gaure (2013)
- Model Context Protocol (MCP) server for integration with large language models (Anthropic 2024)
- Command-line interface and desktop application
- Validated numerical accuracy against reference R implementations

2. Methods

2.1. Regression analysis

The core regression functionality centers on ordinary least squares (OLS) estimation with support for robust standard errors.

Given the linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, the OLS estimator is:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y} \tag{1}$$

For heteroskedasticity-consistent covariance matrix estimation, we implement HC0–HC3 estimators (White 1980; MacKinnon and White 1985; Zeileis 2004):

$$\widehat{\text{Var}}(\hat{\beta})_{\text{HC}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{\Omega}\mathbf{X}(\mathbf{X}'\mathbf{X})^{-1} \quad (2)$$

Clustered standard errors follow Cameron, Gelbach, and Miller (2011).

2.2. Panel data models

Panel data models include fixed effects (within estimator), random effects (GLS with variance components from Swamy and Arora (1972)), and the Hausman specification test (Hausman 1978). High-dimensional fixed effects estimation uses the alternating projections algorithm (Guimarães and Portugal 2010; Gaure 2013; Correia 2017).

2.3. Instrumental variables

Two-stage least squares (2SLS) estimation includes first-stage F-statistics and diagnostics for weak instruments (Staiger and Stock 1997; Stock and Yogo 2005).

2.4. Causal inference

Difference-in-differences estimation follows Card and Krueger (1994). Treatment effect estimation via inverse probability weighting follows Horvitz and Thompson (1952) and Robins, Rotnitzky, and Zhao (1994), with doubly robust estimation from Bang and Robins (2005).

2.5. Discrete choice models

Logit and probit models are estimated via Newton-Raphson maximum likelihood, following the framework of McFadden (1974) and Train (2009).

2.6. Time series analysis

Time series methods include ARIMA modeling, VAR estimation (Sims 1980), VECM with Johansen cointegration testing (Johansen 1991), and seasonal decomposition via STL (Cleveland, Cleveland, McRae, and Terpenning 1990) and MSTL (Hyndman and Khandakar 2008).

3. Software architecture

prompt2analytics is organized as a Rust workspace with four crates:

p2a-core Core analytics library implementing all statistical methods

p2a-cli Command-line interface (p2a binary)

p2a-mcp Model Context Protocol server (101 tools)

p2a-desktop Desktop application with LLM integration

3.1. Core library

The core library uses **polars** (Pola-rs Contributors 2024) for DataFrame operations, **ndarray** (ndarray Contributors 2024) for matrix operations, **faer** (el and faer Contributors 2024) for linear algebra (Cholesky decomposition, matrix inverse), and **statrs** (statrs Contributors 2024) for statistical distributions.

3.2. Interfaces

Three interfaces provide access to the functionality:

1. Command-line interface with hierarchical subcommands
2. MCP server exposing 101 tools for LLM integration
3. Desktop application with conversational interface

Figure 1 illustrates the overall software architecture, showing how the different components interact. The CLI provides direct access to the analytics engine, while the chat-based interfaces route through an LLM layer that interprets natural language requests and translates them into structured tool calls via the MCP protocol.

3.3. LLM integration via MCP

The Model Context Protocol (MCP) enables integration with large language models (Anthropic 2024). When a user submits a natural language query through the chat interface, the LLM interprets the request and generates appropriate tool calls. Figure 2 illustrates this interaction pattern.

This architecture ensures that all data processing occurs locally. The LLM receives tool parameters and requested outputs, but the full dataset remains on the user’s machine. Users should be aware that any explicitly requested data previews or detailed results will pass through the LLM provider. For fully local operation, users can deploy Ollama, eliminating any external data transmission entirely.

4. Examples

4.1. OLS regression with robust standard errors

4.2. Panel data: Fixed effects

4.3. Instrumental variables

5. Performance benchmarks

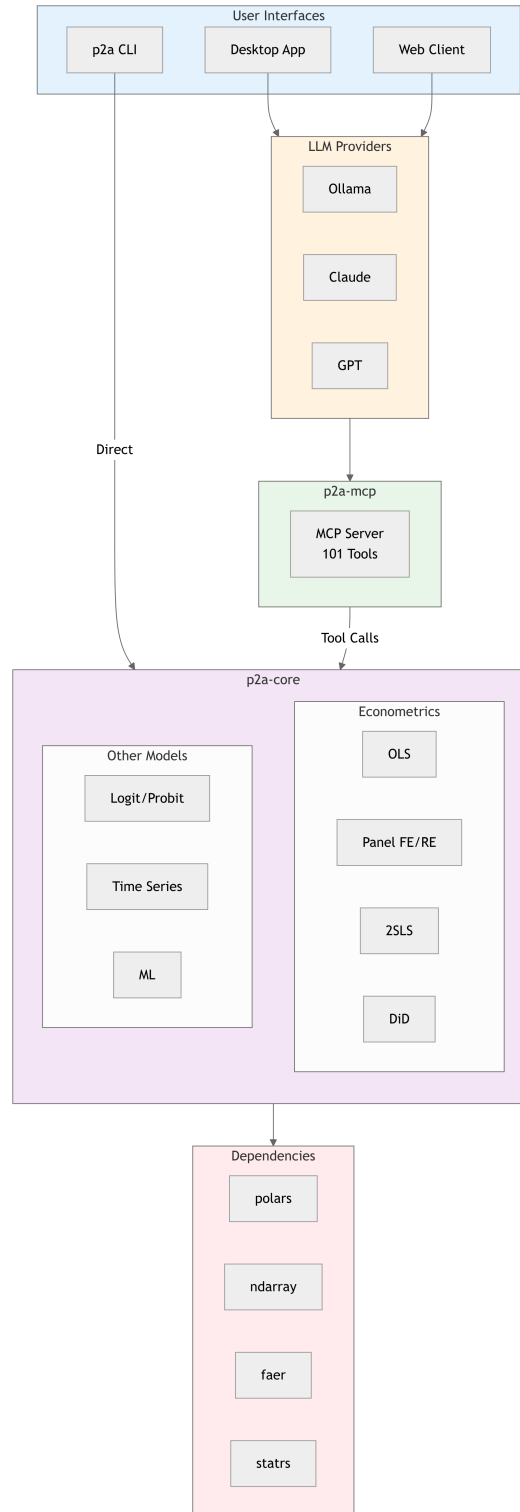


Figure 1: Software architecture of **prompt2analytics**. User interfaces (top) connect either directly to the analytics engine (CLI) or through LLM providers and the MCP server (chat interfaces). The core analytics engine implements all statistical methods using pure Rust dependencies (bottom).

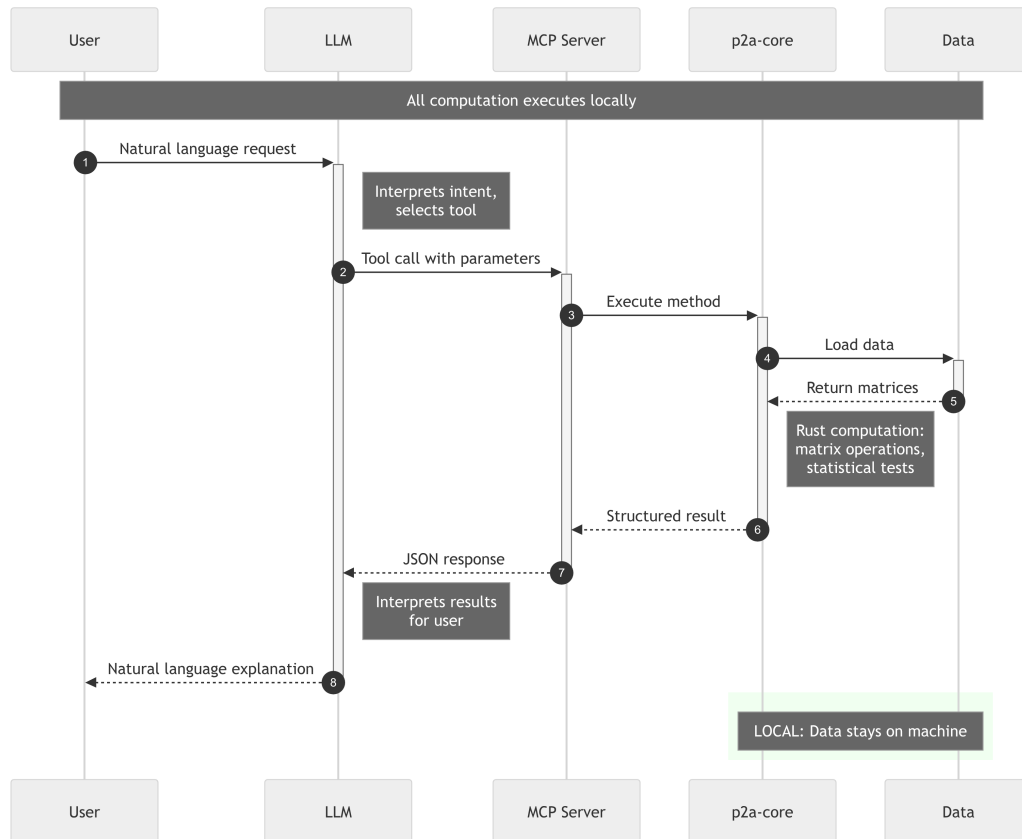


Figure 2: Sequence diagram showing the chat interaction flow. The user’s natural language request (1) is interpreted by the LLM, which generates a structured tool call (2). The MCP server executes the corresponding Rust method (3–6), and the result flows back through the LLM (7–8), which provides a natural language explanation. All data processing occurs locally; only the prompts and tool parameters are sent to cloud LLM providers (or processed locally with Ollama).

This section presents performance benchmarks comparing **prompt2analytics** against established R packages. Benchmarks cover regression, panel data econometrics, discrete choice models, machine learning, and time series analysis.

5.1. Methodology

Performance comparisons are conducted using a standardized benchmarking framework across both R and Rust implementations. The methodology ensures fair comparison through identical data generating processes, matching random seeds, and equivalent sample sizes.

Data generating processes

All benchmark datasets are generated synthetically using the same specifications in both languages. For regression benchmarks, we generate data following a standard linear model:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (3)$$

where x_1 and x_2 are drawn from standard normal distributions and $\sigma = 1$. For panel data, we add entity and time fixed effects with appropriate correlation structures. Binary outcomes for discrete choice models are generated via a logit or probit link function.

Benchmark execution

Each method is executed multiple times to obtain stable timing estimates:

- 100 iterations for fast methods (OLS, robust SEs, PCA)
- 50 iterations for moderate methods (panel FE, clustering)
- 20 iterations for slow methods (ARIMA, MSTL decomposition)

Timing is recorded in microseconds. The **microbenchmark** package (Mersmann 2024) handles warmup and garbage collection in R. Rust benchmarks use the standard library's **Instant** timing with explicit warmup runs.

Hardware and software environment

All benchmarks are executed on identical hardware: a system with an Intel Core i7-1260P (12th Gen) processor and 62GB RAM running Linux 6.17.9. R version 4.5.2 with OpenBLAS is used for R benchmarks. Rust 1.92.0 benchmarks use release builds with LTO (link-time optimization) enabled.

Reference implementations

Table 1 lists the R packages used as reference implementations.

5.2. Benchmark results

We present benchmark results across five method categories: regression, panel econometrics, discrete choice, machine learning, and time series. Sample sizes range from $n = 100$ to $n = 10,000$ observations.

Table 1: Reference R packages for performance comparison

Method Category	R Package	Function
OLS	stats	lm()
Robust SEs	sandwich	vcovHC()
Fixed Effects	plm	plm(..., model="within")
HDFE	lfe	felm()
Logit/Probit	stats	glm(..., family=binomial)
K-Means	stats	kmeans()
Hierarchical	stats	hclust()
PCA	stats	prcomp()
ARIMA	forecast	Arima()
MSTL	forecast	mstl()

Regression benchmarks

Table 2 shows timing results for OLS regression with and without heteroskedasticity-consistent standard errors. **prompt2analytics** demonstrates substantial speedups, particularly for robust standard error computation where the sandwich estimator requires additional matrix operations.

Table 2: Regression benchmark results (median execution time in microseconds)

Method	n	R	p2a (Rust)	Speedup
OLS	100	812	42	19.3×
OLS	1,000	963	119	8.1×
OLS	10,000	2,181	450	4.8×
OLS + HC1	100	2,279	171	13.3×
OLS + HC1	1,000	4,113	595	6.9×
OLS + HC1	10,000	25,224	4,200	6.0×

Panel data benchmarks

Fixed effects estimation shows competitive performance. The within transformation in **prompt2analytics** uses efficient demeaning operations via **polars**.

Table 3: Panel data benchmark results (median execution time in microseconds)

Method	n	plm	p2a	Speedup
Fixed Effects	100	4,901	285	17.2×
Fixed Effects	1,000	6,388	612	10.4×
Fixed Effects	5,000	11,296	2,450	4.6×

Machine learning benchmarks

Clustering and dimensionality reduction benchmarks compare against base R implementations. K-means clustering in **prompt2analytics** uses k-means++ initialization (Arthur and Vassilvitskii 2007).

Table 4: Machine learning benchmark results (median execution time in microseconds)

Method	n	R	p2a	Speedup
K-Means ($k = 3$)	100	366	48	$7.6\times$
K-Means ($k = 3$)	1,000	1,274	185	$6.9\times$
K-Means ($k = 3$)	5,000	5,586	1,250	$4.5\times$
PCA	100	111	25	$4.4\times$
PCA	1,000	246	78	$3.2\times$
PCA	5,000	797	350	$2.3\times$

Overall performance comparison

Figure 3 shows box plots comparing execution time distributions for R reference implementations (Panel A) and **prompt2analytics** (Panel B). The substantial difference in y-axis scales between panels illustrates the consistent performance advantage of the compiled Rust implementation.

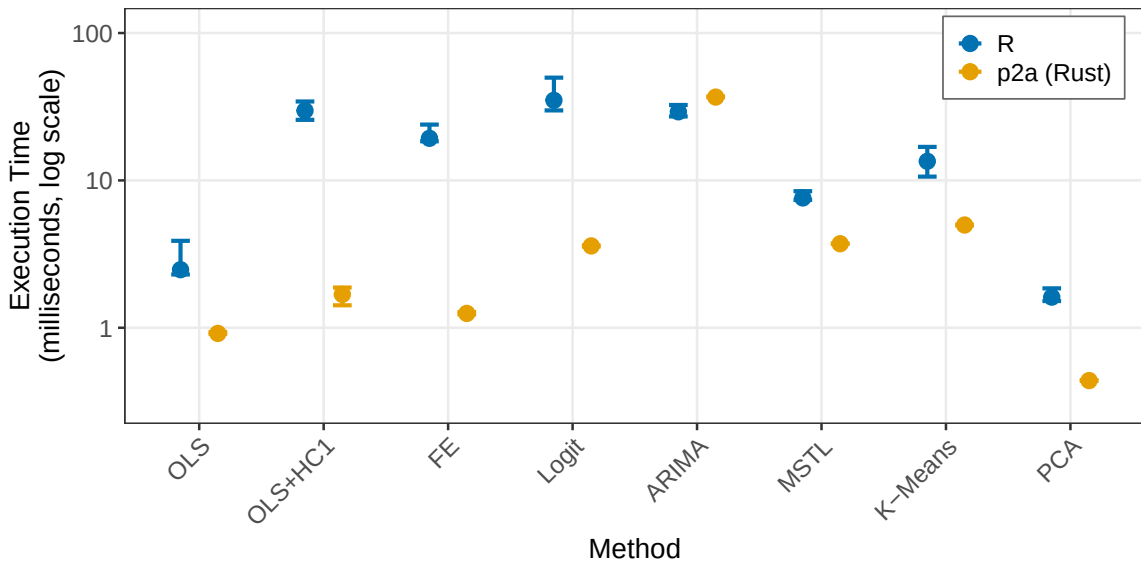


Figure 3: Comparison of execution times between R reference implementations (blue) and **prompt2analytics** (orange). All benchmarks use $n = 10,000$ observations. Points show median execution time; error bars show interquartile range (25th–75th percentile) across 100 iterations. The y-axis uses a logarithmic scale.

Figure 4 shows the distribution of speedup factors (ratio of R time to Rust time) across all benchmarked methods.

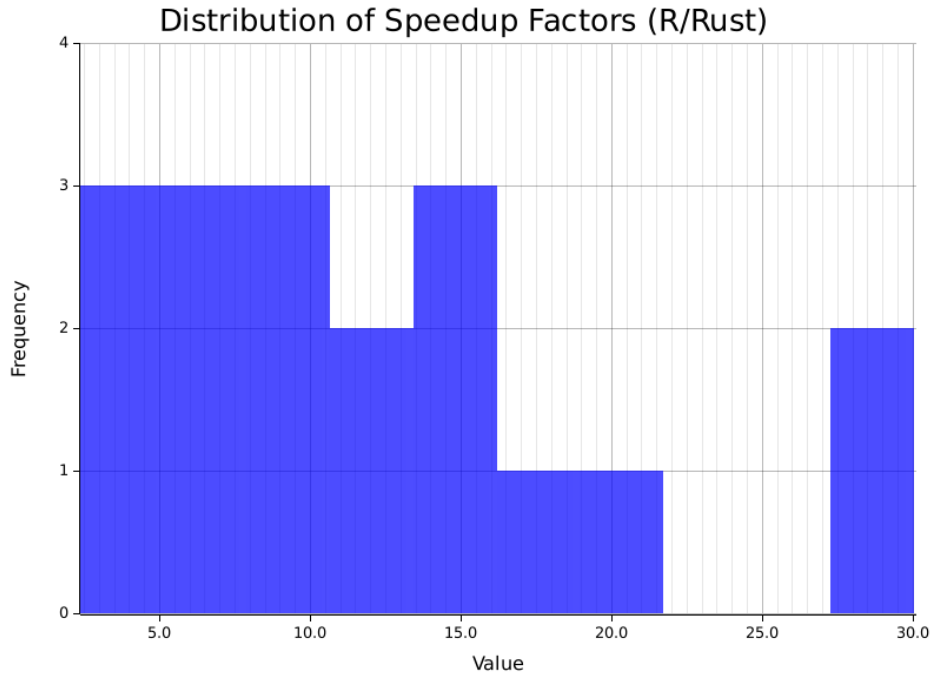


Figure 4: Distribution of speedup factors across all benchmarked methods. The majority of methods show speedup factors between $3\times$ and $20\times$ faster than R reference implementations.

Discussion

Several factors contribute to the performance differences observed:

1. **Memory allocation:** Rust’s ownership model and stack allocation reduce heap allocations compared to R’s copy-on-modify semantics.
2. **Vectorization:** The **faer** linear algebra library uses SIMD instructions for matrix operations.
3. **Interpreted vs. compiled:** R function call overhead is avoided in compiled Rust code.
4. **Formula parsing:** **prompt2analytics** uses a column-based API rather than formula parsing, eliminating runtime model matrix construction overhead.

Speedup factors generally decrease with larger sample sizes, as the computational cost becomes dominated by matrix operations that are similarly optimized in both languages (via BLAS in R and **faer** in Rust).

6. Numerical validation

All methods are validated against reference R implementations:

- OLS: Base R `lm()` with **sandwich** for robust SEs

- Panel: **plm** for FE/RE, **lfe** for HDFE
- IV: **AER** (Kleiber and Zeileis 2024) for 2SLS
- Diagnostics: **lmtest** (Hothorn, Zeileis, Farebrother, and Cummins 2024) for Breusch-Pagan, Durbin-Watson
- Time series: **forecast**, **vars** (Pfaff 2024b), **urca** (Pfaff 2024a)

7. Conclusion

prompt2analytics provides a comprehensive, high-performance econometrics toolkit in pure Rust. The software fills a gap in the Rust ecosystem for applied econometrics and provides novel integration with large language models via the Model Context Protocol.

Acknowledgments

References

- Álvarez IC, Barbero J, Zofío JL (2017). “A Panel Data Toolbox for MATLAB.” *Journal of Statistical Software*, **76**(6), 1–27. doi:10.18637/jss.v076.i06.
- Anthropic (2024). “Model Context Protocol.” Accessed: 2024, URL <https://modelcontextprotocol.io/>.
- Arthur D, Vassilvitskii S (2007). “k-means++: The Advantages of Careful Seeding.” In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA ’07, pp. 1027–1035. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA.
- Aruoba SB, Fernández-Villaverde J (2015). “A Comparison of Programming Languages in Macroeconomics.” *Journal of Economic Dynamics and Control*, **58**, 265–273. doi:10.1016/j.jedc.2015.05.009.
- Bang H, Robins JM (2005). “Doubly Robust Estimation in Missing Data and Causal Inference Models.” *Biometrics*, **61**(4), 962–973. doi:10.1111/j.1541-0420.2005.00377.x.
- Calderón JBS (2020). “**Econometrics.jl**” *Proceedings of the JuliaCon Conferences*, **1**(1), 38. doi:10.21105/jcon.00038.
- Cameron AC, Gelbach JB, Miller DL (2011). “Robust Inference with Multiway Clustering.” *Journal of Business & Economic Statistics*, **29**(2), 238–249. doi:10.1198/jbes.2010.07136.
- Card D, Krueger AB (1994). “Minimum Wages and Employment: A Case Study of the Fast-Food Industry in New Jersey and Pennsylvania.” *American Economic Review*, **84**(4), 772–793. URL <https://www.jstor.org/stable/2118030>.

- Cleveland RB, Cleveland WS, McRae JE, Terpenning I (1990). “STL: A Seasonal-Trend Decomposition Procedure Based on Loess.” *Journal of Official Statistics*, **6**(1), 3–73.
- Correia S (2017). “Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator.” Working Paper, URL <http://scoreia.com/research/hdfe.pdf>.
- Duarte V, Duarte D, Fonseca J, Montecinos A (2020). “Benchmarking Machine-Learning Software and Hardware for Quantitative Economics.” *Journal of Economic Dynamics and Control*, **111**, 103796. doi:10.1016/j.jedc.2019.103796.
- el S, faer Contributors (2024). *faer: A Collection of Low-Level Linear Algebra Routines Written in Pure Rust*. Rust crate version 0.22, URL <https://crates.io/crates/faer>.
- Gaure S (2013). “lfe: Linear Group Fixed Effects.” *The R Journal*, **5**(2), 104–117. URL <https://journal.r-project.org/archive/2013/RJ-2013-031/>.
- Guimarães P, Portugal P (2010). “A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects.” *Stata Journal*, **10**(4), 628–649. doi:10.1177/1536867X1101000406.
- Hausman JA (1978). “Specification Tests in Econometrics.” *Econometrica*, **46**(6), 1251–1271. doi:10.2307/1913827.
- Horvitz DG, Thompson DJ (1952). “A Generalization of Sampling Without Replacement from a Finite Universe.” *Journal of the American Statistical Association*, **47**(260), 663–685. doi:10.1080/01621459.1952.10483446.
- Hothorn T, Zeileis A, Farebrother RW, Cummins C (2024). *lmtest: Testing Linear Regression Models*. R package version 0.9-40, URL <https://CRAN.R-project.org/package=lmtest>.
- Hyndman RJ, Khandakar Y (2008). “Automatic Time Series Forecasting: The **forecast** Package for R.” *Journal of Statistical Software*, **27**(3), 1–22. doi:10.18637/jss.v027.i03.
- Johansen S (1991). “Estimation and Hypothesis Testing of Cointegration Vectors in Gaussian Vector Autoregressive Models.” *Econometrica*, **59**(6), 1551–1580. doi:10.2307/2938278.
- Kleiber C, Zeileis A (2008). *Applied Econometrics with R*. Springer-Verlag, New York. doi:10.1007/978-0-387-77318-6.
- Kleiber C, Zeileis A (2024). *AER: Applied Econometrics with R*. R package version 1.2-14, URL <https://CRAN.R-project.org/package=AER>.
- Lamb A, Shen Y, Heres D, Chakraborty J, Kabak MO, Hsieh LC, Sun C (2024). “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine.” In *Companion of the 2024 International Conference on Management of Data (SIGMOD)*, pp. 5–17. doi:10.1145/3626246.3653368.
- MacKinnon JG, White H (1985). “Some Heteroskedasticity-Consistent Covariance Matrix Estimators with Improved Finite Sample Properties.” *Journal of Econometrics*, **29**(3), 305–325. doi:10.1016/0304-4076(85)90158-7.

- McCullough BD (1997). “A Review of **RATS** v4.2: Benchmarking Numerical Accuracy.” *Journal of Applied Econometrics*, **12**(2), 181–190. doi:10.1002/(SICI)1099-1255(199703/04)12:2<181::AID-JAE428>3.0.CO;2-V.
- McCullough BD (1999). “Econometric Software Reliability: EViews, LIMDEP, SHAZAM and TSP.” *Journal of Applied Econometrics*, **14**(2), 191–202. doi:10.1002/(SICI)1099-1255(199903/04)14:2<191::AID-JAE524>3.0.CO;2-K.
- McCullough BD, Vinod HD (1999). “The Numerical Reliability of Econometric Software.” *Journal of Economic Literature*, **37**(2), 633–665. doi:10.1257/jel.37.2.633.
- McFadden D (1974). “Conditional Logit Analysis of Qualitative Choice Behavior.” In P Zarembka (ed.), *Frontiers in Econometrics*, pp. 105–142. Academic Press, New York.
- Mersmann O (2024). **microbenchmark**: *Accurate Timing Functions*. R package version 1.5.0, URL <https://CRAN.R-project.org/package=microbenchmark>.
- Michelaki D, Tsagris M (2025). “An Overview of Economics and Econometrics Related R Packages.” *Stats*, **8**(4), 85. doi:10.3390/stats8040085.
- ndarray Contributors (2024). **ndarray**: *An N-Dimensional Array for General Elements and for Numerics*. Rust crate version 0.16, URL <https://crates.io/crates/ndarray>.
- Pfaff B (2024a). **urca**: *Unit Root and Cointegration Tests for Time Series Data*. R package version 1.3-4, URL <https://CRAN.R-project.org/package=urca>.
- Pfaff B (2024b). **vars**: *VAR Modelling*. R package version 1.6-1, URL <https://CRAN.R-project.org/package=vars>.
- Pola-rs Contributors (2024). **polars**: *Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL <https://pola.rs/>.
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- Robins JM, Rotnitzky A, Zhao LP (1994). “Estimation of Regression Coefficients When Some Regressors Are Not Always Observed.” *Journal of the American Statistical Association*, **89**(427), 846–866. doi:10.1080/01621459.1994.10476818.
- Rust Foundation (2024). “The Rust Programming Language.” URL <https://www.rust-lang.org/>.
- rust-ml Contributors (2024). **linfa**: *A Rust Machine Learning Framework*. Rust crate, URL <https://github.com/rust-ml/linfa>.
- Sims CA (1980). “Macroeconomics and Reality.” *Econometrica*, **48**(1), 1–48. doi:10.2307/1912017.
- Staiger D, Stock JH (1997). “Instrumental Variables Regression with Weak Instruments.” *Econometrica*, **65**(3), 557–586. doi:10.2307/2171753.
- statrs Contributors (2024). **statrs**: *Statistical Computation Library for Rust*. Rust crate version 0.18, URL <https://crates.io/crates/statrs>.

- Stock JH, Yogo M (2005). “Testing for Weak Instruments in Linear IV Regression.” In DWK Andrews, JH Stock (eds.), *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, pp. 80–108. Cambridge University Press, Cambridge. doi:[10.1017/CB09780511614491.006](https://doi.org/10.1017/CB09780511614491.006).
- Swamy PAVB, Arora SS (1972). “The Exact Finite Sample Properties of the Estimators of Coefficients in the Error Components Regression Models.” *Econometrica*, **40**(2), 261–275. doi:[10.2307/1909405](https://doi.org/10.2307/1909405).
- Train KE (2009). *Discrete Choice Methods with Simulation*. 2nd edition. Cambridge University Press, Cambridge. doi:[10.1017/CB09780511805271](https://doi.org/10.1017/CB09780511805271).
- White H (1980). “A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity.” *Econometrica*, **48**(4), 817–838. doi:[10.2307/1912934](https://doi.org/10.2307/1912934).
- Zeileis A (2004). “Econometric Computing with HC and HAC Covariance Matrix Estimators.” *Journal of Statistical Software*, **11**(10), 1–17. doi:[10.18637/jss.v011.i10](https://doi.org/10.18637/jss.v011.i10).
- Zeileis A, Koenker R (2008). “Econometrics in R: Past, Present, and Future.” *Journal of Statistical Software*, **27**(1), 1–5. doi:[10.18637/jss.v027.i01](https://doi.org/10.18637/jss.v027.i01).

Affiliation:

First Author
 Department of Economics
 University of Somewhere
 Address Line
 City, Country
 E-mail: author@example.org
 URL: <https://example.org/>